# Holistic Root Cause Analysis for Data Quality Issues

## A Lineage-Adaptive Framework

*A comprehensive methodology for identifying, attributing, and resolving data quality defects across enterprise data ecosystems — with or without lineage metadata.*

Version 1.0 | February 2026

# 1. The Problem: Why RCA for Data Quality Is Hard

When a data quality rule fires, the detection is only the beginning. The far harder question is: where did the defect originate, what caused it, and how do we prevent it from recurring? This is Root Cause Analysis (RCA) for data quality.

The difficulty arises from three compounding factors:

**Pipeline complexity:** Modern data environments are directed acyclic graphs (DAGs) with dozens of transformation stages. A single field in a dashboard may trace through 5+ joins, aggregations, and type conversions across multiple systems.

**Lineage gaps:** Formal lineage metadata is a luxury. Many organizations have partial lineage, outdated catalogs, or no lineage at all. Legacy systems, manual data entry, and third-party feeds are often black boxes.

**Multi-causal nature:** Data quality failures rarely have a single cause. A defect may result from a schema change interacting with a code deployment interacting with a new data source — a combinatorial explosion of possibilities.

> **⚠ The Core Design Principle**
>
> This framework is lineage-adaptive: it uses lineage when available but never requires it. Instead, it layers six independent evidence-gathering techniques that together triangulate the root cause. The more signals available, the higher the confidence is, but even a single signal layer produces actionable hypotheses.

# 2. Framework Overview: The Six-Layer RCA Method

The framework operates as a layered evidence funnel. Each layer gathers a different class of signal, and the outputs are synthesized into a ranked set of root cause hypotheses. The layers are designed to be independent — any subset can produce useful results.

| | Layer | Core Question | Prerequisite | Availability |
|---|---|---|---|---|
| **Layer 1** | **Defect Characterization** | What exactly is wrong? | Always available | ☑ Always |
| **Layer 2** | **Temporal Analysis** | When did it start? | Always available | ☑ Always |
| **Layer 3** | **Segmentation Analysis** | Who/what is affected? | Always available | ☑ Always |
| **Layer 4** | **Statistical Profiling** | What patterns co-occur? | Always available | ☑ Always |
| **Layer 5** | **Change Event Correlation** | What changed? | Needs event log access | ⚠ Conditional |
| **Layer 6** | **Lineage Traversal** | Where in the pipeline? | Needs lineage metadata | ⚠ Conditional |

**Key insight:** Layers 1–4 require nothing but the data itself. No lineage, no metadata catalog, no pipeline access. This means the framework works on any dataset, including flat files received from third parties, legacy system extracts, or data lakes with no governance.

# 3. Layer 1: Defect Characterization

***Core question:*** What exactly is wrong, and what is the nature of the defect?

Before investigating causes, you must precisely classify the symptom. A vague description like "data is wrong" is useless for RCA. Defect characterization produces a structured defect fingerprint that drives all subsequent analysis.

## 3.1 Defect Fingerprint Schema

| Attribute | Question It Answers | Example |
|---|---|---|
| **DQ Dimension** | Which quality dimension is violated? | Accuracy, Validity, Completeness, Consistency, Uniqueness, Timeliness |
| **Affected Field(s)** | Which specific columns fail? | CelsiusTemperatureQuantity, RelativeHumidityNumber |
| **Failure Pattern** | What does the bad data look like? | Values > 100, null clusters, format mismatch, duplicated keys |
| **Failure Volume** | How many records are affected? | 42 of 40,000 rows (0.105%) |
| **Failure Distribution** | Concentrated or scattered? | All 42 rows from station 761220-00999 |
| **First Observed** | When was it first detected? | 2026-02-15T08:30:00Z (rule monitoring) |
| **Severity** | Business impact classification | Critical / High / Medium / Low |

## 3.2 Defect Taxonomy

Every defect falls into one of these mechanistic categories, which narrows the hypothesis space for root cause:

| Type | Definition | Example | Likely Cause Class |
|---|---|---|---|
| **Corruption** | Valid data was altered | Temperature was 20.5, now 2050 | Transformation bug, unit conversion error, decimal shift |
| **Injection** | Invalid data was introduced | UUID field contains "INVALID-UUID" | Source system error, test data leak, migration artifact |
| **Omission** | Required data is missing | Entire column is null for batch | Schema change, dropped column, failed extraction |

| | | | |
|---|---|---|---|
| **Duplication** | Records appear more than once | Same station+timestamp, 3 copies | Retry logic, idempotency failure, merge error |
| **Drift** | Data gradually deviates from norm | Mean temperature rising 0.2°/month | Sensor calibration, slow schema evolution, changing source |
| **Staleness** | Data is not current | Latest record is 72 hours old | Pipeline failure, schedule change, upstream delay |
| **Contradiction** | Crossfield logic violated | Sunset before sunrise, min > max | Timezone bug, field swap, partial update |

### Why This Matters

A Corruption defect points toward transformation logic. An Injection defect points toward source systems.
An Omission defect points toward schema or extraction. Classifying the defect type immediately halves
the search space for root causes — even without any lineage information.

# 4. Layer 2: Temporal Analysis

***Core question:*** When did the defect first appear, and what is its temporal pattern?

Time is the single most powerful RCA signal that requires zero lineage. If you can determine when a defect started, you can correlate it with events that happened at that time.

## 4.1 Onset Detection

The goal is to find the inflection point — the moment when the data transitioned from healthy to unhealthy. Methods:

| Technique | How It Works | Prerequisites & Notes |
|---|---|---|
| **Historical Rule Replay** | Re-run the failing DQ rule against historical snapshots or partitions. Walk backward in time until the rule passes. | Requires: historical data or time-partitioned tables. Works without lineage. |
| **Distribution Shift Detection** | Compare statistical profiles (mean, stddev, null rate, distinct count) across time windows. Flag the window where the profile changes. | Requires: timestamp column. Use Kolmogorov-Smirnov or Jensen-Shannon divergence. |
| **Volume Anomaly** | Chart record counts per time period. A sudden drop/spike often coincides with defect introduction. | Requires: timestamp column. Simple but effective. |
| **Schema Archaeology** | Examine table DDL history, column addition/removal dates, type changes. Correlate with defect onset. | Requires: database metadata (information_schema). No lineage needed. |

## 4.2 Temporal Patterns

The shape of the defect over time is diagnostic:

| Pattern | What It Looks Like | Suggests |
|---|---|---|
| **Step Function** | No defects, then sudden onset that persists | One-time event: deployment, migration, config change, new source onboarded |
| **Spike Pattern** | Sudden burst of defects, then returns to normal | Transient event: network glitch, retry storm, one-off manual correction |
| **Gradual Drift** | Defect rate slowly increases over weeks/months | Continuous process: sensor degradation, accumulating tech debt, slowly changing source |
| **Periodic/Cyclical** | Defects appear at regular intervals | Scheduled process: batch job, monthly report, timezone-sensitive logic |
| **Random Scatter** | Defects appear unpredictably across time | Stochastic cause: race condition, intermittent upstream failure, user error |

# 5. Layer 3: Segmentation Analysis

***Core question:*** Is the defect universal, or concentrated in a specific segment of the data?

If a defect affects all records equally, the cause is likely systemic (a global transformation or schema issue). If it affects only a specific segment, the cause is tied to that segment — a particular source, region, time window, or data type. Segmentation is the fastest way to narrow root cause hypotheses without any lineage.

## 5.1 Segmentation Dimensions

Slice the failing records by every available categorical and temporal dimension, then compute the defect rate for each slice:

| Dimension | How to Slice | Interpretation |
|---|---|---|
| **Source / Origin** | Group by source_system, feed_id, api_version, file_name | If 100% of defects come from Source X, the root cause is at or before Source X |
| **Geography / Region** | Group by country, region, city, timezone | If only one region is affected, suspect region-specific logic or a local source |
| **Time Window** | Group by date, hour, batch_id, partition | If defects cluster in one batch, suspect that batch's processing run |
| **Entity / Key** | Group by station_id, customer_id, product_code | If one entity is disproportionately affected, suspect entity-specific data or config |
| **Data Type / Category** | Group by record_type, status, weather_description | If only "Rain" records fail, suspect category-specific business logic |
| **Processing Stage** | Group by etl_stage, load_type (full/incremental) | If only incremental loads fail, suspect the delta/merge logic |

### The Power of Segmentation Without Lineage

Even if you have zero lineage, discovering that "all 42 failing rows come from source_system=API_v3, loaded on 2026-01-15, for station 761220-00999" gives you a precise root cause hypothesis: Something about the API_v3 onboarding on Jan 15 for that station introduced the defect.This is often sufficient to resolve the issue without ever looking at a lineage graph.

# 6. Layer 4: Statistical Profiling & Cross-Field Analysis

***Core question:*** What other data anomalies co-occur with the defect, and what correlations exist between fields?

This layer moves beyond the failing field itself to examine the entire record. If humidity is wrong, what else is wrong in the same rows? Co-occurring anomalies point toward shared causes.

## 6.1 Techniques

| Technique | Method | Diagnostic Value |
|---|---|---|
| **Co-Anomaly Detection** | For the set of failing rows, profile every other column. Compare distributions against healthy rows. Flag columns where the failing-row distribution differs significantly. | If failing rows also have unusual lat/lon values, suspect a geographic or source-level issue, not a field-specific one. |
| **Cross-Field Correlation** | Compute pairwise correlations between all numeric fields. If two fields that should be independent suddenly correlate in failing rows, a transformation may have coupled them. | Temperature and humidity suddenly perfectly correlated in failing rows = likely a copy/paste bug or formula error. |
| **Pattern Frequency Analysis** | For string fields, extract regex patterns and compare pattern distributions between healthy and failing rows. | Failing rows show pattern \d{5}-\d{4} vs expected \d{6}-\d{5} for station_id = zero-padding was lost. |
| **Null Co-occurrence Matrix** | Build a matrix of which fields are null together. Clusters of co-null fields often trace to a single source or extraction step. | If sunrise, sunset, and UV index are always null together, they likely share a common source or extraction path. |
| **Value Domain Comparison** | Compare the set of distinct values in failing rows against the healthy set. New values appearing only in failing rows are injection candidates. | Failing rows contain WeatherDescription="scattred clouds" (typo) that never appears in healthy data. |
| **Benford's Law / Distribution Shape** | For numeric fields, compare the observed distribution shape against the expected one (e.g., Benford's first-digit law, normal distribution, uniform). | If a field expected to follow a normal distribution suddenly shows bimodal behavior, suspect data from two different sources being merged. |

## 6.2 The Healthy vs. Failing Comparison Paradigm

The most powerful lineage-free technique is to treat the failing rows as a "case" group and the healthy rows as a "control" group, then run a differential analysis:

**1. Isolate** the exact set of rows that fail the DQ rule.

**2. Profile both groups:** For every column, compute summary stats (min, max, mean, null rate, distinct count, top patterns) for failing rows and healthy rows separately.

**3. Compute deltas:** Rank columns by how much their profiles differ between the two groups. The columns with the largest deltas are the most diagnostic.

**4. Interpret:** The top-ranked columns reveal what is different about the failing records — and this difference is either the cause or a strong correlate of the cause.

# 7. Layer 5: Change Event Correlation

***Core question:*** What changed in the environment around the time the defect appeared?

This layer requires access to operational metadata — deployment logs, schema change histories, configuration management, and similar event sources. It does not require data lineage.

## 7.1 Change Event Taxonomy

| Event Type | Sources | Investigation |
|---|---|---|
| **Code Deployment** | Git commits, CI/CD pipelines, ETL code changes, stored procedure updates | Check: Were any transformation scripts modified around the defect onset? Tools: Git log, deployment manifests, Airflow DAG version history |
| **Schema Change** | Column additions/removals, type changes, constraint modifications, DDL history | Check: Did any table in the pipeline undergo ALTER TABLE near the onset? Tools: information_schema, schema migration logs, Liquibase/Flyway |
| **Configuration Change** | Connection strings, API endpoints, credential rotations, feature flags | Check: Did a config change redirect data from a different source? Tools: Config management systems, environment variable audit logs |
| **Source System Change** | New data source onboarded, vendor API version change, upstream schema migration | Check: Did the upstream provider change their data format or encoding? Tools: API changelog, vendor release notes, contract SLAs |
| **Infrastructure Event** | Server migration, cloud region change, storage tier change, network interruption | Check: Did an infra event cause data loss, reordering, or encoding change? Tools: CloudWatch, Datadog, PagerDuty incident history |
| **Data Volume Anomaly** | Unusual record counts, unexpected full-load instead of incremental, backfill jobs | Check: Did a backfill or re-extraction job run that introduced stale or duplicate data? Tools: Job scheduler logs, Airflow task instances |
| **Human Intervention** | Manual data corrections, ad hoc SQL updates, spreadsheet uploads, manual overrides | Check: Did someone manually edit data in production? Tools: Database audit logs, query history, change tracking tables |

## 7.2 Correlation Method

Once you have the defect onset time from Layer 2, scan all change event sources for events in a time window around the onset (typically ±24–72 hours). Rank candidate events by:

**Temporal proximity:** Closer in time to the onset = higher suspicion.

**Scope overlap:** If the change affects the same table, column, or pipeline that contains the defect = higher suspicion.

**Segment match:** If the change is specific to the same segment identified in Layer 3 = much higher suspicion.

# 8. Layer 6: Lineage Traversal (When Available)

***Core question:*** At which specific stage in the data pipeline was the defect introduced?

When formal lineage metadata exists, it provides the most precise signal for pinpointing the injection point. However, even without a formal lineage catalog, there are techniques that approximate lineage.

## 8.1 With Formal Lineage

**Boundary Testing:** Starting from the point of detection, walk upstream through the lineage graph. At each stage boundary, sample data and re-run the failing DQ rule. The first stage where the rule passes marks the injection point — the defect was introduced between that stage and the next.

**Transformation Code Inspection:** Once the injection stage is identified, examine the transformation code (SQL, Spark, ETL mappings) for logic that could produce the observed defect. An LLM can assist by reading the code and correlating it with the defect pattern.

## 8.2 Without Formal Lineage: Approximation Techniques

When no lineage metadata exists, these techniques can reconstruct approximate lineage:

| Technique | How It Works | Strengths & Limitations |
|---|---|---|
| **Column Name Matching** | Scan all accessible tables for columns with the same or similar names. Build a candidate lineage graph based on naming conventions. | Fast. High recall, low precision. False positives from coincidental names. Best for narrow, domain-specific column names. |
| **Value Overlap Analysis** | For the failing column, compute the distinct value set and search for tables that contain matching values. High overlap = likely upstream source. | More precise than name matching. Computationally expensive on large catalogs. Can use sampling to reduce cost. |
| **Schema Fingerprinting** | Generate a "fingerprint" for each table (column count, column types, column name set). Cluster tables with similar fingerprints as likely stages. | Identifies tables that are transformations of each other. Misses aggregation stages where schema changes dramatically. |
| **Query Log Mining** | Parse database query logs (query_history, pg_stat_statements) to find which tables are read/written by the same queries or jobs. | Reconstructs actual data flow from usage patterns. Requires query log access. Most accurate lineage-free approach. |
| **Timestamp Chain Analysis** | If multiple tables contain overlapping data with timestamps, compare record-level timestamps to infer data flow direction (source has earlier create_timestamp than target). | Simple, intuitive. Works when tables share a common key and timestamp. Cannot detect intermediate transformations. |

| | | |
|---|---|---|
| **Metadata Annotation Mining** | Search table/column comments, tags, wiki pages, Confluence docs, dbt model descriptions for mentions of upstream/downstream relationships. | Captures institutional knowledge. Incomplete and often outdated. Best as a supplementary signal. |

### Query Log Mining: The Hidden Lineage Goldmine

In most database systems, query logs are enabled by default but rarely used for lineage. A query like "INSERT INTO warehouse.weather_clean SELECT ... FROM staging.weather_raw" explicitly encodes a lineage edge. Mining a month of query logs can reconstruct 80–90% of the actual data lineage without any formal lineage tool. This is the single most underused technique in the industry.

# 9. Synthesis: Hypothesis Generation & Ranking

Each layer produces evidence. The synthesis stage combines all evidence into a ranked list of root cause hypotheses.

## 9.1 Evidence Matrix

Construct a table mapping each hypothesis to the evidence from each layer that supports or contradicts it:

| Hypothesis | L1: Defect Type | L2: Temporal | L3: Segment | L4: Statistical | L5: Change | L6: Lineage |
|---|---|---|---|---|---|---|
| **API v3 sends pre-multiplied humidity** | Corruption ✓ | Step @ Jan 15 ✓ | Only API_v3 ✓ | Co-anomaly in source_type ✓ | API v3 onboarded Jan 15 ✓ | N/A (no lineage) |
| **ETL unit conversion bug** | Corruption ✓ | Step @ Jan 15 ✓ | Only API_v3 ✓ | No co-anomaly ✗ | No ETL deploy Jan 15 ✗ | N/A |
| **Sensor hardware failure** | Drift ✗ | Not gradual ✗ | Only API_v3 ✓ | Isolated field ✗ | No infra event ✗ | N/A |

## 9.2 Confidence Scoring

Each hypothesis receives a confidence score based on:

**Supporting layers:** Each layer that provides confirming evidence adds +1 to the score (max 6).

**Contradicting layers:** Each layer that provides contradicting evidence subtracts −1.

**Layer weight:** Lineage traversal (Layer 6) gets 2x weight because it provides the most direct evidence. Segmentation (Layer 3) gets 1.5x because segment isolation is highly diagnostic.

**Specificity bonus:** Hypotheses that explain all observed symptoms (not just some) get a +1 bonus.

In the example above, Hypothesis 1 ("API v3 sends pre-multiplied humidity") scores 5/5 supporting layers with zero contradictions, making it the clear winner even without lineage.

# 10. Validation & Confirmation

A hypothesis is not a conclusion until it is validated. Validation techniques depend on the hypothesis type:

| Method | Approach | Strength |
|---|---|---|

| | | |
|---|---|---|
| **Reproduce the Defect** | Take the hypothesized cause and deliberately apply it to clean data. Does it produce the same defect pattern? | Strongest validation. If applying the hypothesis to healthy data recreates the exact failure, confidence is near-certain. |
| **Fix and Verify** | Apply the hypothesized fix to a sample of failing data. Does it resolve the defect? | Practical and fast. If the fix works, the hypothesis is confirmed by reversal. |
| **Counterfactual Test** | If the hypothesis is "Source X is the cause," check: does the defect appear in data not from Source X? | Eliminates false correlations. If defects also appear in non-Source-X data, the hypothesis is incomplete. |
| **A/B Comparison** | Compare two batches: one processed before the hypothesized cause and one after. The defect should appear only in the "after" batch. | Requires temporal access to before/after data. Very convincing when available. |
| **Expert Review** | Present the hypothesis, evidence matrix, and sample data to a domain expert or data steward. | Catches logical errors. Essential for business-logic hypotheses that statistics alone cannot confirm. |

# 11. Remediation & Prevention

Once the root cause is confirmed, the final step is a three-part remediation:

**1. Immediate Fix:** Correct the current bad data. This may involve re-running a pipeline, applying a data patch, reverting a code change, or contacting the upstream source.

**2. Preventive Control:** Add a DQ rule that specifically detects this failure mode in the future. The rule should be as close to the injection point as possible, not just at the final consumption layer.

**3. Structural Improvement:** Address the systemic condition that allowed the defect. This may involve adding schema validation at ingestion, implementing contract testing with upstream providers, or adding idempotency checks to ETL logic.

## 11.1 The RCA Knowledge Base

Every completed RCA should be recorded in a knowledge base with the following structure:

| Field | Content |
|---|---|
| **Defect Fingerprint** | The full Layer 1 classification (dimension, fields, pattern, volume, severity) |
| **Onset Time & Pattern** | When it started and the temporal shape (step, spike, drift, periodic) |
| **Affected Segment** | The precise segment (source, region, entity, batch) that was impacted |
| **Root Cause Statement** | One sentence: "The API v3 source sends humidity as a 0–1 fraction while the ETL multiplies by 100, producing values 0–100x too high." |

| | |
|---|---|
| **Evidence Summary** | Which layers supported the conclusion and the key evidence from each |
| **Remediation Applied** | What was fixed, when, and by whom |
| **Preventive Rule Added** | The DQ rule ID and specification that now guards against recurrence |
| **Time to Resolution** | How long from detection to confirmed fix |

### The Flywheel Effect

Over time, the RCA knowledge base becomes a pattern library. When a new defect matches a known fingerprint, the system can suggest probable root causes from historical resolutions. This transforms RCA from a manual, expert-dependent process into a semi-automated, knowledge-driven one — and it works regardless of whether lineage metadata exists.

## 12. Quick-Reference Decision Matrix

Use this matrix to determine which layers to prioritize based on what is available in your environment:

| Scenario | L1 | L2 | L3 | L4 | L5 | L6 |
|---|---|---|---|---|---|---|
| **Flat file from third party, no metadata** | ✓ | ✓* | ✓ | ✓ | ✗ | ✗ |
| **Database table with timestamps** | ✓ | ✓ | ✓ | ✓ | ⚠ | ✗ |
| **Warehouse with query logs, no catalog** | ✓ | ✓ | ✓ | ✓ | ✓ | ⚠** |
| **Full lineage + data catalog** | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| **Streaming pipeline, real-time** | ✓ | ⚠ | ✓ | ⚠ | ✓ | ⚠ |

*✓ = Fully available ⚠ = Partially available ✗ = Not available*

** If the file has a timestamp column ** Query log mining can reconstruct approximate lineage*

## 13. Implementation Roadmap

Building this framework incrementally:

| Phase | Scope | Description | Estimated Effort |
|---|---|---|---|
| **Phase 1** | **Defect + Temporal + Segment** | Implement Layers 1–3. Requires only data access. Produces actionable hypotheses for ~60% of DQ issues. | 2–4 weeks |
| **Phase 2** | **Statistical Profiling** | Add Layer 4. Automate healthy-vs-failing comparison. Catches cross-field and co-anomaly patterns. | 3–6 weeks |
| **Phase 3** | **Change Correlation** | Add Layer 5. Integrate with Git, CI/CD, schema migration, and job scheduler APIs. | 4–8 weeks |
| **Phase 4** | **Lineage Integration** | Add Layer 6. Connect to lineage catalog (Informatica, OpenLineage, etc.) OR implement query log mining. | 6–12 weeks |
| **Phase 5** | **Knowledge Base + ML** | Build the RCA knowledge base. Train a classifier to match new defect fingerprints to historical root causes. | 8–16 weeks |